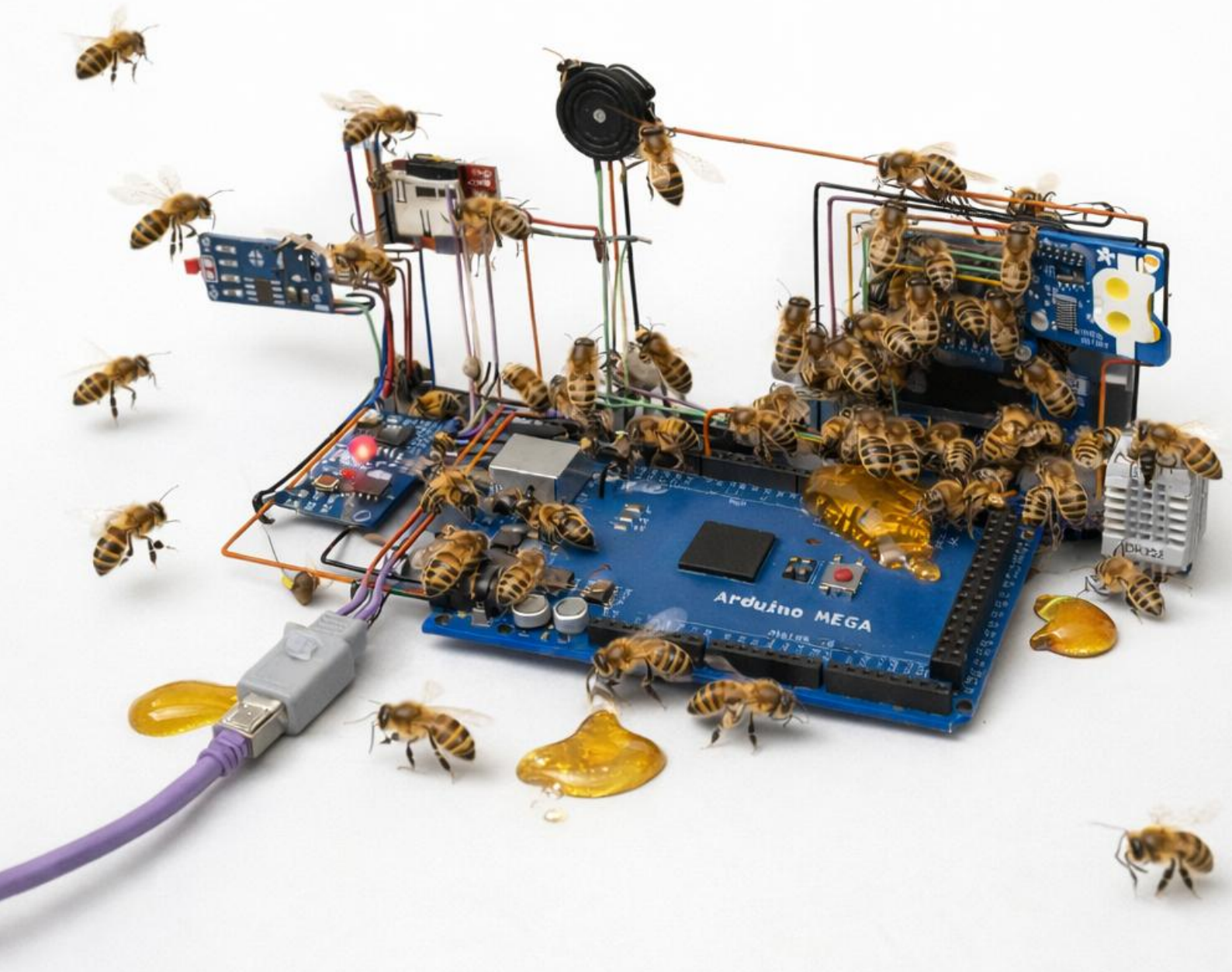
Projekt Beehive Datalogger

Tomasz Blaszczyk, Zuhair Haroon Khan & Palle Zych

Indlejrede Systemer, Programmering, Projektstyring & forretningsforståelse



Indhold

Indledning inkl. problemformulering 3
Problemformulering 3
Metode 3
Analyse 4
Systemarkitektur 4
Flowchart 4
Krav analyse 5
Sensorkrav og Arduino-setup 5
Programstruktur og OOP-implementering 7
Dataanalyse 9
Databehandling 9
Visualisering af data 9
Mikrofon 10
Fejlkilder 10
Vurdering 10
Interessentanalyse 11
Interne interessenter 11
Eksterne interessenter 11
Vurdering 12
Dokumentation af udviklingsproces Scrum 12
Idégenerering sprint 12
Sprint 1 13
Sprint 2 16
Sprint 3 17
Konklusion 18
Dokumentation af kode 19
Kildeliste 37
Bilag 39

Indledning inkl. problemformulering

Biavlere benytter i dag ofte manuelle metoder til at overvåge tilstanden i deres bistader. Denne form for overvågning kan være tidskrævende og indebærer samtidig en risiko for at forstyrre bierne unødigt.
Da bistader ofte er placeret væk fra boligen, kan det være vanskeligt for biavleren løbende at holde øje med miljøforholdene. Derfor kan der være behov for en løsning, som kan give et overblik over bistadets tilstand uden konstant fysisk tilstedeværelse.
I projektet arbejdes der med udviklingen af et produkt, der automatisk kan indsamle og gemme data om miljøforhold i et bistade. Formålet er at give biavleren et grundlag for at vurdere, om der er behov for handling, baseret på målbare data frem for manuel kontrol.

På den baggrund opstilles følgende problemformulering.

Problemformulering

Hvordan kan vi udvikle et system, der hvert 10. minut måler temperatur, luftfugtighed, lysniveau & lydniveau i et bistade, tidsstempler og gemmer data i et struktureret filformat på et SD-kort, samt præsenterer og meddeler dem for biavleren, så data kan bruges til løbende overvågning og efterfølgende analyse af bistadets tilstand?

Metode

Vi valgte at skrive vores projekt i objektorienteret programmering(OOP). Dette valgte vi fordi, vi ville have en struktureret og modulær opsætning af kode. Det giver fordele såsom overskuelighed og legoklods struktur, så hvert komponents opsætning i kode kan skrives eller redigeres uden at ændre i de resterende komponenters kode. I den sammenhæng har vi brugt unit-testing på hver komponent, inden de er blevet sammensat, hvilket har sparet os tid ved udelukkelse af potentielle fejlkilder. Vores objektorientering kode er opdelt i cpp og header filer. Vi har desuden valgt at bruge Github (se kilde 18) som vores platform for data, da det giver en nem synkronisering og opstilling af filer og kode.
Vi har benyttet Trello som vores KanBan-board til at kunne holde overblik, prioritere og uddele opgaver for at kunne nå vores mål.

Scrum master blev valgt og sprints blev planlagt både opgavemæssigt og i tidsrammer. Vi brugte sprint planlægning, udførte sprint, arbejdede iterativt, koordinerede løbende og fik forbedret vores metoder baseret på sprint review, evaluering og backlogs.

Analyse

Systemarkitektur

Systemet er opbygget som et uafhængigt IoT-system, hvor arduinoen fungerer som den centrale enhed (se bilag 4). Sensorer til temperatur, luftfugtighed, lys og lyd leverer data til arduinoen, som står for at aflæse, behandle og validere målingerne. Hver måling bliver tidsstemplet ved hjælp af et RTC-modul og gemt på et SD-kort i CSV-format.

OLED-displayet bruges som lokal visning og statusindikator. Her kan man se, om systemet kører korrekt, samt om der opstår fejl på RTC- eller SD-modulet, uden at det er nødvendigt at tilslutte en computer. Denne arkitektur er valgt for at sikre en simpel og stabil løsning, som kan fungere selvstændigt i et bistade uden netværksforbindelse.

Opbygningen gør systemet modulær, så sensorer og funktioner kan udskiftes eller udvides uden at ændre den overordnede struktur i systemet. Samtidig betyder lokal datalogning, at kompleksiteten holdes nede, og at der er færre potentielle fejlkilder sammenlignet med en netværksbaseret løsning.

Flowchart

Flowchartet viser det overordnede forløb i programmet fra opstart til løbende logning. Programmet starter i setup, hvor kommunikation oprettes, OLED-displayet initialiseres først, derefter SD-kortet og til sidst RTC-modulet. Denne rækkefølge er valgt, så systemets status og eventuelle fejl kan vises på OLED-displayet, selv hvis et af de efterfølgende moduler ikke starter korrekt.

Efter opstart går programmet ind i hovedloopet. Her oprettes et datasæt, hvorefter temperatur og luftfugtighed aflæses fra begge sensorer, og lysværdien registreres. Temperaturmålingerne valideres for at sikre, at de ligger inden for et realistisk interval. Hvis en måling er ugyldig, registreres en fejl via Serial Monitor, og programmet venter 2 sekunder, før der forsøges igen.

Når gyldige temperaturmålinger er opnået, udskrives data til Serial Monitor. Herefter måles lydniveauet fra mikrofonen, og lydniveauet klassificeres. Lysværdien anvendes samtidig til at afgøre, om det er dag eller nat. Derefter hentes et tidsstempel fra RTC-modulet.

De samlede data logges herefter på SD-kortet. Efter skrivning kontrolleres det, om data er gemt korrekt. Hvis skrivningen lykkes, sættes SD-status til OK, og hvis den fejler, registreres SD-status som fejl. Tidsstemplet opdeles i dato og tid, så det kan vises mere overskueligt.

OLED-displayet opdateres med de nyeste målinger og systemets aktuelle status. Hvis SD-kortet ikke er aktivt, vises en fejlmeddelelse på displayet. Hvis alt fungerer som det skal, vises data i normal visning. Til sidst afventer programmet et delay på 10 minutter, hvorefter loopet starter forfra, og der foretages en ny måling og logning.

Denne struktur sikrer, at der kun logges gyldige data, at fejl kan opdages under drift, og at systemet kan køre stabilt over længere tid uden bruger griber ind (se bilag 3).

Krav analyse

Vores produkt skal kunne udføre følgende krav for at logge aktivt(opgavekrav + gruppens valg):

- Skrive på SD kort hvert 10. minut (tidsstempel)
- Aflæse temperatur inde og ude
- Aflæse fugtighed inde og ude
- Aflæse lys
- Aflæse lyd
- Meddele biavler

Sensorkrav og Arduino-setup

Vi skal have adskillige sensorer, som kan måle fugtighed, temperatur, lys og lyd-niveau på bistadet. Temperatur og fugtighed skal både kunne måles ude og inde af bistadet, og det hele sættes sammen via et Arduino Mega2560.

Temperatur og Fugtighed (DHT22)

For temperatur og fugtighed, har vi valgt at gå med en DHT22 sensor. Sensoren kommunikerer med arduinoen via et digitalt signal. Den gør vores krav om temperatur og fugtighed både inde og ude opfyldt, da denne sensor har begge sensorer i ét modul med

digital kommunikation. Vi valgte at bruge DHT22 fremfor DHT11 fordi DHT22 er mere nøjagtig i både temperatur og fugtighed, samt den har en større range. DHT11 har en range på 0-50 grader celsius, så den er ikke optimal til temperaturmåling udenfor i december måned. Derfor er DHT22 mere optimal til vores projekt da den har en range på -40 til 80 grader celsius(se kilde 6 og 7).

Lys-sensor (photoresistor - LDR)

Med lys-sensoren har vi valgt at gå med en simpel photoresistor, som blot skal have en Analog to Digital Converter (ADC) på 10-bit, så får vi en værdi som vi kan bruge(0-1023). Vi skal blot kalibrere den så vi ved præcis hvilken værdi der svarer til nat og hvilken der svarer til dag (se kilde 13)

Mikrofon (KY-037)

Biernes aktivitet kan måles på hvor høj lyd de laver. F.eks. hvis de brummer højt kan det være et tegn på stress i bistadet. Det kan enten betyde at der er forstyrrelse af bierne, at de skal til at sværme, at der er 2 dronninger eller hvis dronningen er død. Vi valgte at gå med KY-037 mikrofonen.
En af de ideer vi kom med under idegenerering, grundet vi havde en nær kontakt til en nuværende biavler, var at vores datalogger skulle kunne meddele brugeren om hændelser i bistadet. Så de havde mulighed for at reagere, inden det skete. Dette kunne for eksempel være sværme. Denne hændelse er ingen interesse for biavlerne, og derfor tænkte vi over hvad vi ville kunne gøre ved dette. Den mikrofon vi har anvendt i projektet er analog, og sender et spændingssignal som aflæses af arduinoens ADC. Den leverer ikke en præcis måling med decibel, men den leverer vores egen måleenhed, hvilket stadig kan bruges til tracking af bierne. En mere avanceret løsning man kunne have gjort i stedet for, for at levere dataen i decibel som er en mere genkendelig måleenhed, var at lave en estimering af decibel omregning ud fra vores system. Det ville give en mere detaljeret analyse af biernes adfærd. Det fravalgte vi, da vi skulle kalibrere med ekstern modul, som vi ikke kunne anskaffe os i tide. Dette holdt systemet simpelt samt fokuserer på stabil datalogning (se kilde **Fejl! Henvisningskilde ikke fundet.**).

SD logging (Micro-SD card adapter)

Vi er blevet undervist i skrivning på SD-kort med en Micro-SD card adapter i CSV-format. Det tager vi til os og bruger aktivt i vores produkt, med modulet som skriver data fra andre sensorer ned på fil. Modulet kommunikerer med arduinoen via et digitalt signal (se kilde 14).

Tidsstempel (RTC module - DS3231)

Arduino har ikke et internt clock-modul, så vi må tage et modul udefra. Her har vi valgt Real Time Clock (RTC), som vi har erfaring med fra undervisning. Vores krav om et timestamp (ISO 8601), opfyldes med RTC modulet. RTC kommunikerer med arduinoen via et digitalt signal (se kilde 15).

Meddele til bruger (SSD1306)

Kravet om at vores produkt skulle kunne meddele til bruger havde vi tænkt som en notifikation eller mail-service. Der blev undersøgt på mulige services, hardware og løsninger. En indikator fra dataloggeren som fortalte den stadig loggede, er en god ide. Det kan vi gøre via en LED lampe som blinker som en markør om at det stadig logger.
Vi har valgt at demonstrere det på et OLED-display(SSD1306 OLED I2C 128x64 Display), da vi ikke har ressourcer eller kompetencer til at oprette server, og det informerer mere end en LED lampe. Vores OLED-display kommer derfor til at virke som vores info-display, som bruger ville kunne åbne op og se data på, samt vigtig information. Den kommer desuden til at hjælpe udviklingen af vores produkt, da vi kan kontrollere selv, når vi lægger produkt til logning, at den er aktiv og læser korrekte værdier. Displayet kommunikerer med arduinoen via et digitalt signal (se kilde 16).

Shell

Vores produkt skal sidde i et bistade, med et par sensorer uden for bistadet, så nogen form for beskyttelse skal vi have med. Her vil vi gå med et 3D-print af en kasse, hvor vi borer et hul i siden til vores udendørs sensorer. Kassen skal have et låg som kan puttes over, så den får ly fra vejr og vind (Se vedhæftet mappe: 3Dprint).

Programstruktur og OOP-implementering

Programmet er opdelt i separate klasser for hvert modul. At vi opdeler objektorienteret sikrer, at vi kan ændre i én sensor eller modul uden at påvirke andre dele af systemet, hvilket øger modulær fleksibilitet og gør systemet lettere at vedligeholde.

Sensordata

For at holde en ren kode i vores ino-fil, har vi valgt at lave en separat h-fil der indeholder de respektive data. Dette gør at i vi kan nøjes med at have simple kald i inofilen frem for at have variabler stående samt at det gør det mere simpelt både at overskue og udvide hvis ønsket.

Fejlsikring

Før hver logning udføres et while-loop, der sikrer at temperaturmålinger er gyldige. Denne tilgang sikrer, at ingen fejlagtige data logges. Efter validering, tidsstemples det via RTC-modulet og logges på SD.

Datahåndtering og SD-logning

Data logges i en tekstfil via SD-klassen. Hver logning inkluderer temperatur, fugtighed, lysniveau, lyd, dag/nat-status samt tid og dato. For at informere brugeren om hvorvidt at SD-modulet fungerer efter hensigten, vil der i tilfælde af fejlet logning blive der skrevet på OLED at SD ikke er aktivt.

Brugerfeedback og visualisering

OLED-displayet viser alle relevante data, inkl. SD-status. Dette designvalg giver brugeren direkte feedback. Displayet fungerer derfor som en lokal løsning, der demonstrerer, at konceptet virker og potentielt kan udvides til en remote løsning for biavleres bekvemmelighed.

Problematik

Vores løsning på at sikre gyldige data har den begrænsning at hvis den bliver ved med at få fejlmålinger, så vil den forsøge igen 2 sekunder efter og derved potentielt forskyde næste måling i sekvensen.

Sammenfatning

Objektorienteret programmering har gjort det muligt at designe et modulær, overskueligt og robust system, hvor hver komponent kan testes og valideres separat. Datahåndtering sikrer pålidelig måling og lagring, mens OLED-displayet giver direkte brugerfeedback.

Dataanalyse

Vi har skulle demonstrere at vores produkt kan logge data så biavler kan få målbart information, hvilket de kan bruge i bistade for at spare både tid og penge. Vores analyse fokuserer på parametre som lys, lyd, luftfugtighed og temperatur. Data kan behandles ved hjælp af middelværdi, minimum - og maksimumværdier samt grafer visualiseret i Excel. Denne data er logget hvert 10. minut og gemt, i vores tilfælde en .txt-fil på SD-kort. Vores data indeholder, i rækkefølge: Tidsstempel(ISO 8601), Temperatur(inde), Fugt(inde), Temperatur(ude), Fugt(ude), Mikrofon, Dagscyklus(dag/nat), Lys(ren lys værdi).

Databehandling

Middelværdier	Middelværdier er beregnet pr time for både inde og ude sensor.
Minimum og maksimum	Minimum og maksimum værdien af midlet er noteret for hver dag, så bruger ser hvor miljøforhold har været mest ekstreme.
Solminutter	Dagscyklus blev udregnet så vi kunne skelne mellem dag og nat. Sol tid kan da beregnes inden for ~10 minutter.

Visualisering af data

Herunder er data visualiseret inde i Excel på en graf, som giver et overblik over indsamlet data. Alle værdier er vist under graf farvekodet. Selve produktet satte vi i en flamingo-kasse på en altan, hvor vi satte varmelegeme ind af 3 omgange for at påvise temperatur og fugt både inde og ude. Det er derfor der fremkommer 3 ukurante “spikes” i graferne. Samtidig med de temperatur spikes, vises der et fald i den indendørs luftfugtighed som sker ifølge af at opvarmningen sænker luftfugtigheden i kassen. Den udendørs temperatur og luftfugtighed varierer mere jævnt og påvirkes primært af vejrforhold (dog meget svag stigning samtidig med de indsatte varmelegemer) Samlet set viser dataene en klar sammenhæng mellem aktiv opvarmning og ændringer i indeklimaet, mens de udendørs målinger fungerer som reference og understøtter, at udsvingene indendørs skyldes varmelegemet.
Ved en aktiv stade vil der oftest være mellem 15-25 grader med den højeste temperatur værende i midten af stadet ved ”klyngen” som bierne har sat sig i vinterperioden. Dette er på trods af at der kan være/er frost grader udenfor.
I vores måling ville der i en reel situation med et aktivt stade være et potentielt problem. Temperaturen er måske til den lave side, men den kunne overleves. Faren her er fugten, ved høj fugt er der for det første risiko for bierne at de får sværere ved at holde sig tørre og

varme og dermed øge sygdomsrisiko og dødelighed. Et andet problem ville være at med høj fugt/kondens er et optimalt miljø for mug og svampevækst på honning og tavler.
Dog er den store forskel om der er kondens i stadet eller blot høj fugtighed. Så dette er et godt eksempel på hvornår avler selv skulle ud og føre opsyn på stadet (se bilag 7)

Mikrofon

Måling af vores lydniveau har vi valgt at tage separat fra den resterende data, grundet at den under hele vores logging periode udviste “lav aktivitet” eftersom der ikke har været nogen aktivitet at måle pga. mangel på bier.
Dette er blevet simuleret med tanke på normal aktivitet på en almen dag i sommerperioden og vist på følgende graf (se bilag 6)

Som før nævnt kunne man med den rette hardware, ifølge aktiv forskning, udmåle flere indikatorer på eksempelvis: kommende sværm, sygdom eller stress.

Dog giver nuværende opsætning give en indikator på hvorvidt der er aktivitet eller ej og stadig kræve manuelt opsyn ved evt. høj eller lav aktivitet hvis uden for norm på årstid.

Fejlkilder

Under prototype fasen i sprint 1 fik vi fejlaflæsninger fra DHT22(se Sprint 2 evaluering). For at sikre at vi fik korrekt data fra sensorer blev de ugyldige målinger udfaset vha. et while-loop som, hvis den fik data mindre end -50 grader celsgius(hvilket det aldrig bliver), så ventede den 2 sekunder og derefter målte igen. Dette frasorterer fejlkilder, som vi havde tidligere identificeret, fuldstændig.

Vurdering

Vores analyse viser at det produkt vi har udviklet et produkt som aktivt kan opsamle og lagre, ved print på SD-kort, data fra sensorer som kan anvendes af biavlere til overvågning og evt. analyse af biers adfærd og miljø. Ved hjælp af databehandling fandt vi middelværdier, ekstremer og visualisering er det demonstreret at produktet lever op til projektets krav og problemformulering.

Interessentanalyse

I dette projekt er der flere interessenter, som enten har indflydelse på udviklingen af systemet eller bliver påvirket af det færdige produkt. Formålet med interessentanalysen er at skabe overblik over, hvem disse interessenter er, og hvilken rolle de spiller i projektet.

Interne interessenter

Projektgruppen

Projektgruppen er den vigtigste interessent i projektet. Det er gruppen, der står for udvikling, programmering, test og dokumentation af systemet. Alle tekniske beslutninger, valg af hardware og struktur i koden bliver truffet internt i gruppen. Gruppen har derfor både høj interesse og høj indflydelse på projektets resultat.

Vejledere samt undervisere

Underviserne fungerer som vejledere og sparringspartnere undervejs i projektet. De stiller krav til opgavens indhold og sikrer, at projektet lever op til læringsmålene. Samtidig er det underviserne, der vurderer projektet ved eksamen, hvilket giver dem stor indflydelse, selvom de ikke deltager direkte i udviklingsarbejdet.

Medstuderende

Der deles primært erfaringer, problemstillinger og løsninger med de studerende, derfor fungere de som en daglige sparringspartnere. Hvilket hjælper projektgruppen i form af tids og arbejds effektivisering.

Eksterne interessenter

Biavlere

Biavlere er de tiltænkte brugere af systemet og dermed den vigtigste eksterne interessent. De har interesse i at kunne overvåge bistadets miljøforhold uden konstant manuel kontrol. Deres behov har haft betydning for valg af sensorer, logningsinterval og visning af data, så løsningen giver praktisk værdi i hverdagen.

Zealand

Skolen fungerer som rammesætter for projektet ved at fastlægge tidsrammer, krav og eksamensform. Institutionen har ikke direkte indflydelse på de tekniske løsninger, men sætter de overordnede rammer, som projektet skal udføres indenfor.

Eventuelle brugere af analyse og data

De indsamlede data kan bruges til efterfølgende analyse, f.eks. i Excel. Disse brugere har interesse i, at data er struktureret korrekt, tidsstemplet og let at arbejde videre med. De har dog ingen direkte indflydelse på udviklingsprocessen.

Vurdering

Projektgruppen og underviserne er de interessenter med størst indflydelse på projektets gennemførelse og resultat. Biavlere har høj interesse i det færdige produkt, men begrænset direkte indflydelse på udviklingsprocessen. Interessentanalysen har været med til at sikre, at systemet både opfylder de formelle krav til projektet og samtidig skaber en løsning, der er relevant og anvendelig for slutbrugeren.

Dokumentation af udviklingsproces

Scrum

Teamet bruger scrum, hvilket er et kendt tool som mange teams og firmaer bruger til at optimere resultater med forskellige roller og events. Vores product owner er i vores relation skolens vejledere som giver os opgaven. Vi kommer til at planlægge 3 sprints, hvor vi fokuserer på sprint reviews, backlogging og adaptation.

Idégenerering sprint

I dette sprint har vi brainstormet idéer til projektet med fokus på, hvordan det færdige produkt kan hjælpe en biavler. Der blev færdiglavet en problemformulering, som vi udarbejdede fra opgavebeskrivelsen. Vi har diskuteret, hvilke funktioner apparatet skal have, og lavet en prioritering mellem “must have” og “nice to have”.

Need to have	**Nice to have**
Temperaturmåler	Mikrofon

Time tracker	Notifikation
Lys sensor	
Fugt sensor	

Vi har taget inspiration fra biavler Jens-Erik Jensen som har givet os idéer til praktiske løsninger, og vi vil fortsætte med at bruge dette som kilde undervejs i projektet. På baggrund af brainstorm og diskussioner har vi fået et klart billede af, hvad projektet skal indeholde, både i forhold til udstyr, input/output og de funktioner, apparatet skal kunne udføre.

Sprintet var kortfattet og skulle blot sætte rammerne for det færdige produkt, så vi kunne planlægge ud fra det.

Sprint 1

Dette sprint startede med vores opsætning af datalogger og programmering/kodning i Arduino. Vi opsatte en liste over ting som vi skulle udføre inden kl. 14:00 på fredag(5. december), hvor vores device skal afleveres til logging. På listen havde vi én Need to have og 3 Nice to have. Se tabel:

Opgave nr.	**Logger skal kunne**	**Prioritering**
1	Gemme data fra RTC på SD	**NEED** to have
2	Temperatur & Fugtighed	**NICE** to have
3	Lyd(mikrofon)	**NICE** to have
4	Printe data på OLED display	**NICE** to have

Listen er nu opsat og vi begynder på vores objektorienteret kode i Arduino.
Vores DHT22 temperatur og fugtighedssensor havde vi husket svært at få til at virke korrekt, så vi prøvede at sætte den op for at se, om dem som vi havde på hånden overhovedet virkede. Vi havde problemer med at få dem til at vise korrekt data.
Vi forsøgte først at skrive koden om på en masse forskellige måder, hvorefter vi forsøgte forskellige libraries til modulet, og endda flere tidligere versioner af de libraries da det måske var en opdaterings skyld. Det viste sig ikke at være derfor. Tomasz, vores underviser, blev hentet. Han fortalte os at ét af modulerne havde en indbygget resistor, så den måske opførte sig lidt anderledes. Vi forsøgte os med multimeteret på alle pins på begge moduler, men

ingen fejl blev fundet. Vi undersøgte hvordan man kunne finde ud af hvad fejlen var, om det var hardware eller software. Efter noget research fandt vi ud af at man kunne lave en debugging-code(se kilde 22) Vi implementerede debugging koden for at finde ud af hvad problemet reelt var. Det viste sig at der var checksum fejl på vores sensor. Vi satte en if-statement ind, som gjorde at den kun målte når den fik data mellem -50 og +50 grader, og anden data ville gemmes som “NAN”(not a number). Når sensoren faktisk målte, fik den korrekt data. Al anden info var nemlig ugyldigt og ubrugelig. Grundet denne fejl valgte vi at lave en undersøgelse for at finde ud af hvor ofte den forekom. Vi valgte at lave undersøgelsen sådan at vi tog ca 600 datalogs for at illustrere hvor meget data vi ville få hvis vores datalogger kørte 3 dage i træk med logning hvert 10 min. Derfor lavede vi en undersøgelse hvor vi lavede et logging delay på 3 sec og så kørte den i ca 45 min, for at vi nåede det ønskede antal logs. Undersøgelsen viste at dataen for vores dht22 lavede var der 22% fejl i målinger(se bilag 1 for fejldata).
Efter finpudsning ændrede vi den til at den kun skulle logge når den fik korrekt data, og ingen “NAN”. Dette gjorde vi ved at sætte et while-loop ind hvor den kun gemte data, når den kunne få gyldig info, ved at måle igen hvert 2. sekund indtil gyldig data var målbart.

Herefter kunne vi aflæse korrekt. Nu indser vi dog at vi har brugt en del timer af sprinten på vores opgave nr. 2, som jo er en nice to have. Efter det sætter vi en ende på opgave nr. 2 og fokuserer på opgave nr. 1, som vi burde have startet med til at starte med. Vi må nu være agile for at rette op.

Nu begynder vi at sætte SD kortet i vores microSD card modul. Her fik vi byttet om på ground og 5V, så vi formoder at vores modul er brændt af, da det blev meget varmt. Forsøg med multimeter prøvede vi på, og det viste sig at den virkede fint. MicroSD card module fungerer nu korrekt og der kan skrives til CSV/txt fil på SD-kort og vi laver prototype på vores databehandling(får grafer ud fra kort dataopsamling), så vi er klar til når endelig data skal behandles og fremvises.
Senere i sprinten kan RTC ikke kommunikere til SD-kortet, og vi må hælde mange ressourcer og tid ind til fejlfinding og damage control. Ved hjælp af Github har vi branchet ud, og kan derfor blot gå tilbage til virkende kode og se forskellen samt hvor og hvornår fejlen opstod. Det viser sig at der var fejl i software og at vi havde skrevet forkert kode som havde kald på en variabel der var placeret forkert.
Opgave nr. 4 for sprinten er vores OLED display, som blot skal åbnes i ny header og cpp filer, indsætte variabler fra andre moduler og derefter printe dem på skærm. Modulet fungerer, men skal loddes før endelig brug i produkt(se bilag 5). OLED display koden bliver

skrevet færdig og vi kan aflæse info på skærm som en status på logger(se bilag 9).

Evaluering

Sprinten var præget af en del tekniske problemer, både på hardware- og softwaredelen. Vi brugte meget tid på at få tingene til at spille, og flere fejl viste sig først, når vi forsøgte at samle det hele til et færdigt setup. Vi endte derfor ofte med at køre udelukkelsesmetoden for at finde ud af, hvad der reelt var galt.

Hardware

Sensorerne og modulerne opførte sig ikke stabilt fra start, og nogle komponenter skulle fixes eller skiftes. Det kostede tid, men det gav os samtidig en bedre forståelse af, hvad der faktisk sker elektrisk, og hvor små fejl kan skabe store problemer.

Software

Software delen blev en større tidsrøver end forventet. Vi havde delt koden op i h og cpp filer for at få et mere struktureret og professionelt setup. I teorien smart – i praksis skabte det problemer.

Koden virkede enkeltvis, men når vi samlede projektet, holdt det op med at fungere. Vi brugte alt for lang tid på at lede efter logiske fejl, kommunikationen mellem filer og strukturen, før vi til sidst fandt roden til problemet:
Vi manglede et funktionskald i ino-filen, fordi det ved en fejl lå inde i en cpp fil. Arduinoen eksekverede derfor aldrig delen af koden, og projektet virkede kun “halvt”. Da vi endelig fandt det, gav det hele mening, men det viste også, hvor vigtigt det er at have styr på filstrukturen og ikke bare antage, at tingene kobler sig selv sammen.

Læring fra sprinten

- Vi skal tjekke samspillet mellem filer hurtigere i processen, især når vi arbejder med h/cpp filer.
- Funktionskald **skal** ligge de rigtige steder – “det virker hver for sig” er ikke nok.
- Brug udelukkelsesmetoden tidligere, så vi ikke spilder tid på at lede i blinde.
- Start simpelt → udvid strukturen, når vi ved, at baseline virker.

Selvom sprinten var rodet, kom vi igennem det, og vi står nu med et system, der faktisk fungerer – og en langt skarpere forståelse af både vores hardware og vores kodebase.

Sprint 2

Sprint 2 ramte præcis det, vi havde brug for:

Mere struktur, bedre og en klar forventningsstyring. Vi gik ind med én “need to have” og resten som “nice to have”, fordi vi reelt var foran tidsplanen allerede. Det viste sig at være den rigtige beslutning. Gruppen har derudover forbedret kommunikationen internt, som har haft en betydelig positiv effekt på forberedelsen.

Vi kom igennem alle planlagte opgaver undtagen shell-printet, som vi er i gang med nu. Lys-sensoren, mikrofonen, OLED-displayet og hele logging-sekvensen er integreret, og koden er blevet strammet op, så outputtet faktisk ser ordentligt ud. Ingen store problemer undervejs – tingene spillede, og når noget drillede kort, blev det løst hurtigt, fordi vi var mere fokuserede og arbejdede bedre sammen end i Sprint 1.

En af de største forskelle fra sidste sprint: vi planlagde ordentligt fra start. Klare opgaver, klar prioritering, klare stop-punkter.

Der blev arbejdet naturligt mellem os hvor vi separat men stadig samlet fortsatte på OLED-display, hvor vi fik styr på adressering, layout og fik skærmen til at vise sensordata ikke blot data, men som i realtid. Det krævede både tilpasning af libraries og oprydning i output, men resultatet blev et stabilt og læsbart display.

Der blev arbejdet på implementeringen af mikrofonen. Vi testede output sammen og justerede parametre, indtil mikrofonen opførte sig stabilt.

Produktet er nået de krav vi har sat for den for dette sprint og oprydning i koden for simplicitet over et bedre overblik. Her bliver funktioner opdelt, der kommer mere logisk organisering af .hpp/.cpp filer, så der nemmere evt. kan bygges videre på projektet.

Evaluering

Sprint 2 var markant mere effektiv end Sprint 1, og det kom direkte fra det arbejde, vi lagde i planlægningen.

SCRUM-mæssigt lykkedes vi med:

- At bruge prioritering aktivt (Need vs Nice).
- At arbejde iterativt og integrere moduler ét ad gangen.
- At forbedre processen baseret på erfaringer fra Sprint 1.
- At levere en færdig og brugbar version af produktet.

- At holde sprintens scope realistisk og gennemførligt.

Vi havde ingen kritiske tekniske problemer, og når noget drillede fandt vi løsningen hurtigt. Vi samarbejdede bedre, vi kommunikerede mere effektivt og vi havde et klarere billede af hvad næste skridt var.

Dette sprint var et bevis på, at vi lærer af vores fejl. Vi fik styr på koden, outputtet, strukturen og de centrale funktioner i produktet. Vi blev mere professionelle i vores tilgang, og sprinten endte med et samlet, stabilt og fuldt fungerende produkt dog undtaget 3D shell, som vi måtte opgive med tids- og ressourcemangel.

Sprint 3

Vi starter sprintet med planlægning af rapport, med undtagelse af data fra datalogningen. Vi planlægger detaljerne i rapporten, udseende og indhold. Under begyndelsesfasen af rapportskrivning, tjekker vi dataloggeren for kontrol af korrekt data. Her starter problemfasen. Vi opdager at OLED displayet ikke er tændt. Der var en wire som havde fået revet sig løs. Den wire var til RTC'en hvilket betød den ingen strøm fik, og dermed kunne vores proces i koden ikke køre. I og med vi allerede havde tænkt en løsning til lige præcis dette problem, med at vi ikke kunne se på vores datalogger, om den loggede korrekt. Derfor implementerede vi løsningen rimelig hurtigt, da ideen var genereret på forhånd og det eneste mangel var koden. Vores OLED display var sat til at starte op efter RTC-modulet og kunne dermed ikke vise os hvorvidt programmet kørte, da den aldrig kom for RTC.begin i setup. OLED er nu det første der starter op på produktet så vi er sikre på at den kan vise hvad der er galt, frem for at skulle connecte til en PC hver gang for at fejlsøge. Vi lavede en lignende løsning på SD modulet hvor at hvis der er fejl i åbning af logfilen, så vil den skrive det nederst på vores OLED.
Produktet er nu optimeret, og vi vil forsøge en datalogning igen. For at undgå samme problematik har vi valgt at logge data privat. Her måtte vi være kreative for at skabe samme simulerings vilkår som bistadet. Dette blev gjort via en flamingokasse og varmelegeme (se bilag 10) for at skille forskel på inde samt ude temperatur og fugt. Nu bliver der logget. Herefter bliver der skrevet på rapporten og indhold bliver fyldt ud imens vi venter på data. Efter de 3 dages logning, stopper vi dataloggeren og tager SD kortet ud for at behandle data. Dataloggeren har logget efter planen, og vi har brugbart data over den fulde periode. Vi får analyseret, visualiseret og vurderet vores data. For at visualisere data, brugte vi Excel. Vi udarbejdede pivottabeller, som gør det simpelt at lave forskellige diagrammer og grafer.

Evaluering

Fra vores forrige sprints har vi lært at det er vigtigt at planlægge. Dette sprint har lært os hvor vigtigt det er at omprioritere og omstille sig baseret på uforudsete hændelser. Især har vi udviklet os til at løse problemer under tidspres.
Planen gik i vasken ift. datalogning, og vi måtte derfor være omstillingsparate og agile. Vores kreativitet har sparet os for mulige fejlkilder og hjulpet med problemløsning.

Konklusion

Formålet med projektet var at udvikle en datalogger, som automatisk kan måle temperatur, luftfugtighed, lysniveau og lydniveau i et bistade, tidsstemple målingerne og gemme data i et struktureret filformat, så data kan bruges til løbende overvågning og efterfølgende analyse. På baggrund af projektet kan det konkluderes, at denne problemformulering er besvaret.

Den udviklede løsning kan logge miljødata hvert 10. minut, tilføje tidsstempel via RTC-modul og gemme målingerne på et SD-kort i CSV-format. Data kan efterfølgende importeres i Excel og bearbejdes ved hjælp af middelværdier, minimum- og maksimumværdier samt grafer. Dermed viser projektet, at systemet producerer strukturerede og brugbare data, som kan give et overblik over miljøforholdene i og omkring bistadet.

Projektet har samtidig vist, at objektorienteret programmering giver en mere overskuelig og modulær kodebase. Sensorer, datalogning og brugerfeedback kan testes og tilpasses uafhængigt af hinanden, hvilket har gjort fejlfinding og videreudvikling nemmere. Implementeringen af fejlhåndtering har forbedret datakvaliteten ved at sikre, at ugyldige målinger ikke gemmes, hvilket er vigtigt for at undgå misvisende analyser. OLED-displayet fungerer som en lokal statusindikator og gør det muligt at se, om systemet kører korrekt, uden at der er behov for tilslutning til en computer.

Løsningen har dog også begrænsninger. Systemet anvender udelukkende lokal datalogning og understøtter ikke trådløs dataoverførsel eller automatiske notifikationer til brugeren, som vi havde ønsket. Mikrofonen bruges som en relativ indikator for lydaktivitet og leverer ikke præcise målinger i decibel. Disse funktioner kunne være relevante at arbejde videre med i en fremtidig version, hvor systemet eksempelvis udvides med trådløs kommunikation og mere avanceret behandling af lyddata.

Samlet set viser projektet, at det er muligt at udvikle en funktionel og stabil IoT-baseret datalogger, som opfylder projektets krav og demonstrerer grundlæggende kompetencer inden for indlejrede systemer, programmering og projektstyring.

Dokumentation af kode

Datalogger.ino

```
#include "TempSensor.h"
#include "SD_log.h"
#include "RTC.h"
#include "SensorData.h"
#include "Lys_Sensor.h"
#include "Mic.h"
#include "OLED.h"
```

#include gør, at denne fil får adgang til klasser, funktioner og datatyper defineret i andre filer. Hver .h repræsenterer en komponent i systemet (sensorer, SD-logning, RTC og OLED), så koden bliver modulær og lettere at teste og fejlsøge.

```
TempSensor sensor1(7);
TempSensor sensor2(6);
LysSensor ldr(A0);
MikrofonSensor mic(A1);
SD_log sdlog(53, "HiveLog1.txt");
RTC clock1; // dedikerede pins til RTC digital 20 & 21
OledDisplay oled; // SCL & SDA
```

TempSensor sensor1(7) og sensor2(6) opretter to temperatur/fugt-sensor objekter (DHT22), hvor tallet er signalpinnen. Sensorerne aflæses hver for sig, så vi får værdier for både "inde" og "ude".
LysSensor ldr(A0) opretter lys-sensoren på en analog pin. Arduinoens ADC (0–1023) bruges til at måle et relativt lysniveau.
MikrofonSensor mic(A1) opretter mikrofonen på analog pin og bruges til at måle lydamplitude som indikator for aktivitet.

SD_log sdlog(53, "HiveLog1.txt") opretter SD-loggeren med chip-select pin (53 på Mega) og filnavnet, som data gemmes i.
RTC clock1 opretter RTC-objektet, der leverer tidsstempel, da Arduino ikke har intern real-time clock.
OledDisplay oled opretter OLED-displayet, der bruges til status og visning af data uden PC.

```
void setup()
{
    Serial.begin(9600);
    delay(3000);

    oled.begin();
    sdlog.begin();
    clock1.begin();
}
```

Serial.begin(9600) starter seriel kommunikation, så vi kan debugge og se sensorværdier i Serial Monitor.
delay(3000) giver systemet tid til at stabilisere sig efter opstart (f.eks. sensorer/strøm), før moduler initialiseres.
oled.begin() initialiserer OLED-displayet først, så systemet kan vise status/fejl uden at være koblet til en PC.
sdlog.begin() initialiserer SD-modulet og klargør filen til logning.
clock1.begin() initialiserer RTC-modulet, så vi kan hente tid til timestamp.

```
void loop()
{
    SensorData d;
```

Der hentes fra SensorData.h der fungerer som "container" til alle målinger i én loop-cyklus. Det gør det nemmere at holde styr på data og sende det videre til SD-logning og OLED.

```
    while (true)
    {
        d.temp1 = sensor1.readTemperature();
        d.hum1  = sensor1.readHumidity();
        d.temp2 = sensor2.readTemperature();
        d.hum2  = sensor2.readHumidity();
        d.light = ldr.readLight();

        if (d.temp1 > -40 && d.temp2 > -40)
            break;

        Serial.println("Fejl i måling, prøver igen");
        delay(2000);
    }
```

Her aflæses temperatur og fugt fra begge sensorer samt lysniveau.
while(true) bruges som et retry-loop, så der ikke logges ugyldige data.
if (d.temp1 > -40 && d.temp2 > -40) fungerer som en simpel validering, fordi DHT22 typisk returnerer ekstreme/ugyldige værdier ved fejl. Hvis temperaturerne er realistiske, bryder vi ud af loopet.
Hvis målingen er ugyldig, udskrives en fejlbesked til Serial og der ventes 2 sekunder før ny måling. Det forbedrer datakvaliteten ved at undgå "junk values" i logfilen.

```
    Serial.print("Sensor 1 - Temperatur: "); Serial.print(d.temp1, 1);
    Serial.print(" °C, Fugtighed: "); Serial.print(d.hum1, 1);
Serial.println("%");
    Serial.print("Sensor 2 - Temperatur: "); Serial.print(d.temp2, 1);
    Serial.print(" °C, Fugtighed: "); Serial.print(d.hum2, 1);
Serial.println("%");
```

Serial.print() bruges til at vise aktuelle målinger under test og fejlsøgning.

Serial.print(d.temp1, 1) printer med 1 decimal for at gøre output læsbart uden at fylde unødigt meget.

```
    int amplitude = mic.readAmplitude(2000);
    String soundLevel = mic.classifyActivity(amplitude);
```

mic.readAmplitude(2000) måler lydamplitude over en periode (2000 ms) for at få en mere stabil værdi end en enkelt analog-læsning.
mic.classifyActivity(amplitude) oversætter amplituden til et klassificeret niveau (fx lav/medium/høj aktivitet). Det gør output mere brugbart end en rå ADC-værdi.

```
    String dayOrNight = ldr.getDayOrNight();
    Serial.print("Aktivitet: "); Serial.print(soundLevel);
Serial.println(" | " + dayOrNight);
```

ldr.getDayOrNight() bruger lysniveauet til at afgøre om det er "dag" eller "nat" ud fra en grænseværdi.
Resultatet udskrives sammen med lyd-klassificeringen til Serial Monitor.

```
    String timestamp = clock1.getTimestamp();
```

clock1.getTimestamp() henter et tidsstempel fra RTC-modulet (typisk i ISO-8601 format). Tidsstemplet gør det muligt at analysere data over tid og matche målinger til dato/klokkeslæt.

```
    sdlog.log(timestamp, d.temp1, d.hum1, d.temp2, d.hum2, soundLevel,
dayOrNight, d.light);
```

sdlog.log(...) gemmer alle værdier i en fil på SD-kortet i struktureret format.

```
    Serial.print("Tidspunkt: "); Serial.println(timestamp);
```

Serial.println(timestamp) bruges til at verificere at tidsstempling og logning fungerer korrekt.

```
    String dateStr = timestamp.substring(0, 10);
    String timeStr = timestamp.substring(11, 19);
```

substring() bruges til at splitte timestamp til dato og tid, så det kan vises mere overskueligt på OLED-displayet.

```
    String sdStatus = sdlog.isLastWriteOK() ? "OK" : "FAIL";
```

isLastWriteOK() returnerer status på sidste SD-write.
Operatoren ”?” er shortcut for if-statement. Så de 2 muligheder er true eller false, ergo (“OK” eller “FAIL”)

```
    oled.showData(
          dateStr.c_str(),
          timeStr.c_str(),
          d.temp1,
          d.hum1,
          d.temp2,
          d.hum2,
          soundLevel,
          sdStatus);
```

oled.showData(...) opdaterer displayet med dato, tid, sensorværdier, aktivitet og SD-status.
.c_str() bruges fordi OLED-funktionen forventer C-string (char*), mens String i Arduino er et objekt.

```
    delay(600000); // 10 min interval
}
```

delay(600000) pauser programmet i 600.000 ms = 10 minutter. Det sikrer, at systemet logger med det ønskede faste interval.

.h filer

Alle vores header filer er opbygget i ens struktur, derfor forklarer vi header filerne generelt og ikke enkeltvis. Vi har taget udgangspunkt i tempsensor.h i dette eksempel.

```
#pragma once
#include <Arduino.h>
Komponetets tilhørende library
```

#pragma once bruges til at sikre, at TempSensor.h kun bliver inkluderet én gang under kompileringen. Det forhindrer fejl med dobbeltdefinition af klassen, hvis headerfilen inkluderes flere steder i projektet. Arduino.h inkluderes for at give adgang til Arduino-specifikke funktioner og datatyper, som klassen er afhængig af, f.eks. grundlæggende datatyper og timing-funktioner. dhtnew.h er biblioteket, der håndterer kommunikationen med DHT22-sensoren. Biblioteket står for selve sensorprotokollen, timing og checksum-kontrol af de aflæste data.

```
class TempSensor
```

TempSensor er en klasse, der er oprettet for at samle funktionaliteten til temperatur- og fugtighedsmåling i én selvstændig komponent. Klassen gør det muligt at arbejde med sensorerne som objekter og bidrager til en mere modulær og overskuelig kodebase.

```
{
public:
```

public angiver, at de efterfølgende medlemmer i klassen kan tilgås fra andre filer, herunder hovedprogrammet i Datalogger.ino.

```
    DHTNEW sensor;
```

Sensor er et objekt af typen DHTNEW, som repræsenterer den fysiske DHT22-sensor. Objektet bruges til at initialisere sensoren og til at hente temperatur- og fugtighedsdata via DHTNEW-biblioteket.

```
    TempSensor(int pin);
```

TempSensor(int pin) bruges til at oprette et TempSensor-objekt og knytte sensoren til en specifik digital pin på arduinoen. Dette gør det muligt at oprette flere temperatur- og fugtighedssensorer med samme klasse, men på forskellige pins.

```
    float readTemperature();



    float readHumidity();
};
```

readTemperature() og readhumidity() er funktioner, der bruges til at aflæse temperaturen og fugtigheden fra DHT22-sensoren. Funktionerne returnerer værdien som et float, så den nemt kan bruges videre i beregninger, logning og visning.

Tempsensor.cpp

```
#include "TempSensor.h"
```

Her inkluderes TempSensor.h, så implementeringsfilen får adgang til klassens definition, funktionsdeklarationer og sensor-objektet.

```
TempSensor::TempSensor(int pin) : sensor(pin) { }
```

Dette er konstruktøren for klassen TempSensor. Når et TempSensor-objekt oprettes, initialiseres sensor-objektet direkte med den angivne digitale pin. Initialiseringslisten (: sensor(pin)) bruges for at sikre, at DHT-sensoren er korrekt klargjort allerede ved oprettelse af objektet.

```
float TempSensor::readTemperature()
{
    sensor.read();
    return sensor.getTemperature();
}

float TempSensor::readHumidity()
{
    sensor.read();
    return sensor.getHumidity();
}
```

readTemperature() og readHumidity() bruges til at aflæse temperaturen og fugtigheden fra DHT22-sensoren. Først kaldes sensor.read(), som henter nye data fra sensoren og udfører intern validering. Derefter returneres temperaturen og fugtighed via sensor.getTemperature() og sensor.getHumidity() som et float, så værdien kan bruges direkte i logning og visning.

RTC.cpp

```
#include "RTC.h"
```

Her inkluderes RTC.h, så implementeringsfilen får adgang til klassens definition, herunder RTC-objektet, variabler og funktionsdeklarationer.

```
RTC::RTC()
{
}
```

Dette er konstruktøren for klassen RTC. Konstruktøren er tom, da RTC-modulet ikke kræver nogen parametre ved oprettelse. Selve initialiseringen af hardwaren håndteres i begin()-funktionen.

```
void RTC::begin()
```

```
{
    Wire.begin();
    clock.begin();
    Serial.println("RTC klar");
}
```

begin() bruges til at initialisere RTC-modulet. Wire.begin() starter I2C-kommunikationen, som RTC-modulet benytter. clock.begin() initialiserer selve RTC-chippen, og Serial.println("RTC klar") bruges som feedback i Serial Monitor for at bekræfte, at modulet er startet korrekt.

```
String RTC::getTimestamp()
{
    clock.forceConversion();
    dt = clock.getDateTime();
```

getTimestamp() henter et opdateret tidsstempel fra RTC-modulet. clock.forceConversion() tvinger modulet til at opdatere interne værdier, så der arbejdes med den nyeste tid. clock.getDateTime() henter dato og klokkeslæt og gemmer det i variablen dt.

```
    char buffer[25];
    snprintf(buffer, sizeof(buffer),
             "%04d-%02d-%02d %02d:%02d:%02d",
             dt.year, dt.month, dt.day,
             dt.hour, dt.minute, dt.second);
```

Her opbygges tidsstemplet i et fast format ved hjælp af en char buffer og snprintf. Denne metode er mere hukommelses- og pladsbesparende end at sammensætte String-objekter direkte og sikrer samtidig et ensartet format på tidsstemplerne. (Se kilde Fejl! Henvisningskilde ikke fundet.**)**

```
    return String(buffer);
}
```

Den formatterede tekst i bufferen konverteres til en Arduino String og returneres, så tidsstemplet kan bruges direkte til logning, visning på OLED og udskrivning til Serial Monitor.

```
float RTC::getTemperature()
{
    clock.forceConversion();
    return clock.readTemperature();
}
```

getTemperature() bruges til at aflæse RTC-modulets interne temperatur. clock.forceConversion() sikrer, at målingen er opdateret, hvorefter temperaturen returneres som et float. Denne funktion kan bruges til overvågning af modulets temperatur eller som ekstra reference i systemet.

Mic.cpp

```
#include "Mic.h"
```

Her inkluderes Mic.h, så implementeringsfilen får adgang til klassens definition, herunder variabler og funktionsdeklarationer for mikrofonen.

```
MikrofonSensor::MikrofonSensor(int p)
{
    pin = p;
}
```

Dette er konstruktøren for klassen MikrofonSensor. Når objektet oprettes, gemmes den analoge pin, som mikrofonens signal er forbundet til. Det gør klassen fleksibel, da samme klasse kan bruges med forskellige pins.

```
int MikrofonSensor::readAmplitude(int samples)
{
    int mn = 1024;
    int mx = 0;
```

readAmplitude() bruges til at måle lydens amplitude. Variablerne mn og mx initialiseres til henholdsvis den højeste og laveste mulige ADC-værdi, så de kan bruges til at finde minimum og maksimum under målingen.

```
    for (int i = 0; i < samples; i++)
    {
        int val = analogRead(pin);
        if (val < mn) mn = val;
        if (val > mx) mx = val;
    }
```

Her foretages der flere analoge aflæsninger fra mikrofonen. For hver måling opdateres minimums- og maksimumsværdien. Ved at tage mange samples opnås et mere stabilt og repræsentativt billede af lydniveauet end ved en enkelt måling (se kilde 20).

```
    return mx - mn;
}
```

Forskellen mellem maksimum og minimum returneres som amplituden. Denne værdi fungerer som et relativt mål for, hvor kraftig lyden er, uden at forsøge at beregne et absolut lydtryk i decibel.

```
String MikrofonSensor::classifyActivity(int amplitude)
{
```

classifyActivity() bruges til at oversætte en rå amplitudeværdi til en mere meningsfuld tekstbaseret klassifikation, som kan bruges i logning og visning.

```
    if (amplitude < 10)
        return "Lav_aktivitet";
    else if (amplitude < 40)
        return "Normal_aktivitet";
    else
        return "Høj_aktivitet";
```

```
}
```

Her sammenlignes amplituden med faste tærskelværdier. Afhængigt af niveauet returneres en tekststreng, der angiver lav, normal eller høj aktivitet. Denne tilgang gør outputtet mere brugbart end rå tal og giver et hurtigt overblik over lydaktiviteten i bistadet.

Lys_sensor.cpp

```
#include "Lys_Sensor.h"
```

Her inkluderes Lys_Sensor.h, så implementeringsfilen får adgang til klassens definition, variabler og funktionsdeklarationer for lys-sensoren.

```
LysSensor::LysSensor(int p) : pin(p)
{
    pinMode(pin, INPUT);
}
```

Dette er konstruktøren for klassen LysSensor. Den modtager en pin som parameter og gemmer den i variablen pin. pinMode(pin, INPUT) bruges til at konfigurere pinnen som input, så arduinoen kan aflæse det analoge signal fra lys-sensoren korrekt.

```
int LysSensor::readLight()
{
    return analogRead(pin);
}
```

readLight() aflæser lysniveauet fra sensoren ved hjælp af analogRead(pin). Funktionen returnerer en værdi mellem 0 og 1023, som repræsenterer et relativt lysniveau baseret på a kortrduinoens 10-bit ADC.

```
String LysSensor::getDayOrNight()
{
    int value = readLight();
```

getDayOrNight() bruges til at fortolke lysmålingen og afgøre, om det er dag eller nat. Først hentes den aktuelle lysværdi ved at kalde readLight().

```
    return (value < 250) ? "Nat" : "Dag";
}
```

Her sammenlignes lysværdien med en fast grænseværdi på 250. Hvis værdien er under grænsen, klassificeres det som nat, og hvis den er over, klassificeres det som dag. Operatoren "?" er shortcut for if-statement. Så de 2 muligheder er true eller false, ergo ("Nat" eller "Dag")

Oled.cpp

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include "OLED.h"
```

Her inkluderes de nødvendige biblioteker til OLED-displayet.

```
OledDisplay::OledDisplay()
: display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1)
{
}
```

Dette er konstruktøren for klassen OledDisplay. Her initialiseres display-objektet med skærmens bredde og højde samt reference til I2C-kommunikationen. Reset-pinnen sættes til -1, da displayet ikke bruger en dedikeret reset-pin.

```
void OledDisplay::begin() {
    if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
        Serial.println("Skærm virker ikke");
```

```
        for(;;);
    }
```

begin() bruges til at initialisere OLED-displayet. display.begin() starter kommunikationen med displayet. Hvis initialiseringen fejler, udskrives en fejlbesked til Serial Monitor, og programmet stoppes i et uendeligt loop for at forhindre videre kørsel uden visning.

```
    display.clearDisplay();
    display.setTextColor(SSD1306_WHITE);
}
```

Her ryddes displayet, og tekstfarven sættes til hvid. Dette klargør displayet til visning af data.

```
void OledDisplay::showData(const char* date,
                           const char* time,
                           float temp1,
                           float hum1,
                           float temp2,
                           float hum2,
                           String soundLevel,
                           String SD_active)
```

showData() er den centrale funktion til opdatering af OLED-displayet. Funktionen modtager dato, tid, sensorværdier, lydniveau og SD-status som parametre, så displayet kan opdateres samlet ud fra én måling.

```
    display.clearDisplay();
    display.setTextSize(1);
    display.setCursor(0, 0);
```

Displayet ryddes ved hver opdatering, tekststørrelsen sættes til standard, og cursoren placeres øverst til venstre, så teksten skrives fra toppen af skærmen.

```
display.print("Dato:      ");
display.println(date);
display.print("Tid:         ");
display.println(time);
```

Her vises dato og tidspunkt på displayet. Dato og tid modtages som tekststrenge og udskrives, så de er lette at aflæse for brugeren.

```
display.print("Temp. inde:    ");
display.print(temp1, 1);
display.println(" C");
```

Den indendørs temperatur vises med én decimal.

```
display.print("Fugt inde:     ");
display.print(hum1, 1);
display.println(" %");
```

Her vises den indendørs luftfugtighed med én decimal og procentangivelse.

```
display.print("Temp. ude:     ");
display.print(temp2, 1);
display.println(" C");
```

Den udendørs temperatur vises på samme måde som den indendørs, så layoutet er ensartet.

```
display.print("Fugt ude:      ");
display.print(hum2, 1);
display.println(" %");
```

Her vises den udendørs luftfugtighed, igen med samme format som de øvrige værdier.

```
display.print("Lyd:    ");
display.println(soundLevel);
```

Lydniveauet vises som en tekstbaseret klassifikation (fx lav, normal eller høj aktivitet), hvilket gør informationen lettere at forstå end en rå numerisk værdi.

```
    if (SD_active != "OK") {
        display.print("SD IKKE AKTIV");
    }
```

Her kontrolleres status for SD-kortet. Hvis sidste skriveoperation ikke var succesfuld, vises en tydelig fejlmeddelelse på displayet, så brugeren hurtigt kan se, at datalogning ikke fungerer korrekt.

```
    display.display();
}
```

Til sidst opdateres displayet fysisk ved at kalde display.display(). Dette sender al den skrevne tekst til OLED-skærmen og afslutter opdateringen

SD_log.cpp

```
#include "SD_log.h"
```

Her inkluderes SD_log.h, så implementeringsfilen får adgang til klassens definition, variabler og funktionsdeklarationer for SD-logning.

```
SD_log::SD_log(int cs, const char* name)
{
    csPin = cs;
    fileName = name;
    lastWriteSuccess = true;
}
```

Dette er konstruktøren for klassen SD_log. Chip-select-pinnen til SD-modulet gemmes i csPin, og filnavnet til logfilen gemmes i fileName.

Variablen lastWriteSuccess sættes som standard til true og bruges senere til at indikere, om seneste log på SD-kortet lykkedes (se kilde 21).

```
void SD_log::begin()
{
    Serial.print("Indlæser SD... ");
```

begin() bruges til at initialisere SD-modulet. Der udskrives først en statusbesked til Serial Monitor, så det kan ses, at SD-initialisering er i gang.

```
    if (!SD.begin(csPin)) {
        Serial.println("FEJL!");
        lastWriteSuccess = false;
        return;
    }
```

Her forsøges SD-modulet initialiseret med den angivne chip-select-pin. Hvis initialiseringen fejler, registreres fejlen via lastWriteSuccess, og funktionen afsluttes for at forhindre videre logning.

```
    Serial.println("OK");
```

Hvis SD-modulet initialiseres korrekt, udskrives en bekræftelse i Serial Monitor.

```
    File file = SD.open(fileName, FILE_WRITE);
```

Her forsøges logfilen åbnet i skrive-tilstand. Hvis filen ikke eksisterer, oprettes den automatisk.

```
    if (file) {
        file.println("Starter logføring:");
        file.println("Ved fejl skubbes måling 2 sek");
        file.println("Format: Tid | S1 Temp | S1 Fugt | S2 Temp | S2
Fugt");
```

```
        file.close();
```

Hvis filen åbnes korrekt, skrives der en kort header i filen. Headeren bruges som dokumentation direkte i logfilen og gør det nemmere at forstå formatet ved efterfølgende dataanalyse.

```
    } else {
        Serial.println("FEJL: Kan ikke åbne logfil");
        lastWriteSuccess = false;
    }
}
```

Hvis logfilen ikke kan åbnes, registreres fejlen både via Serial Monitor og ved at sætte lastWriteSuccess til false.

```
void SD_log::log(String timestamp, float temp1, float hum1, float temp2,
float hum2, String soundLevel, String dayOrNight, int light)
{
```

log() er den centrale funktion til datalogning. Funktionen modtager tidsstempel, sensorværdier, lydklassifikation, dag/nat-status og lysniveau som parametre.

```
    File file = SD.open(fileName, FILE_WRITE);
    if (!file)
    {
        Serial.println("Fejl i åbning af logfil!");
        lastWriteSuccess = false;
        return;
    }
```

Her forsøges logfilen åbnet igen. Hvis filen ikke kan åbnes, afsluttes funktionen, og fejlen registreres, så systemet kan reagere på det.

```
    file.print(timestamp);
    file.print("|S1Temp:"); file.print(temp1, 1);
```

```
    file.print("|S1Fugt:"); file.print(hum1, 1);
    file.print("%|S2Temp:"); file.print(temp2, 1);
    file.print("|S2Fugt:"); file.print(hum2, 1);
```

Her skrives tidsstemplet og temperatur- og fugtighedsværdier fra begge sensorer til logfilen. Værdierne skrives med fast format og én decimal for at gøre filen ensartet og let at analysere.

```
    file.print("%|");
    file.print(soundLevel + "|");
    file.print(dayOrNight + "|");
    file.print("Lys:");
    file.print(light);
    file.print("\n");
```

Lydaktivitet, dag/nat-status og lysniveau tilføjes til loglinjen. Hver måling afsluttes med linjeskift, så hver logpost fylder én linje i filen.

```
    file.close();
    Serial.println("Data indskrevet på SD");
```

Efter skrivning lukkes filen korrekt for at sikre, at data gemmes fysisk på SD-kortet. En statusbesked udskrives til Serial Monitor.

```
    lastWriteSuccess = true;
}
```

Hvis hele logningsprocessen gennemføres uden fejl, sættes lastWriteSuccess til true, så resten af systemet kan se, at SD-logningen fungerer korrekt.

Kildeliste

1. Margolis, M., Jepson, B., & Weldin, N. R. (2020). *Arduino cookbook* (3rd ed.). O'Reilly.
2. Olsen, N. V., & Lassen, S. M. (2019). *Projektledelse* (1. udg.). Gads Forlag

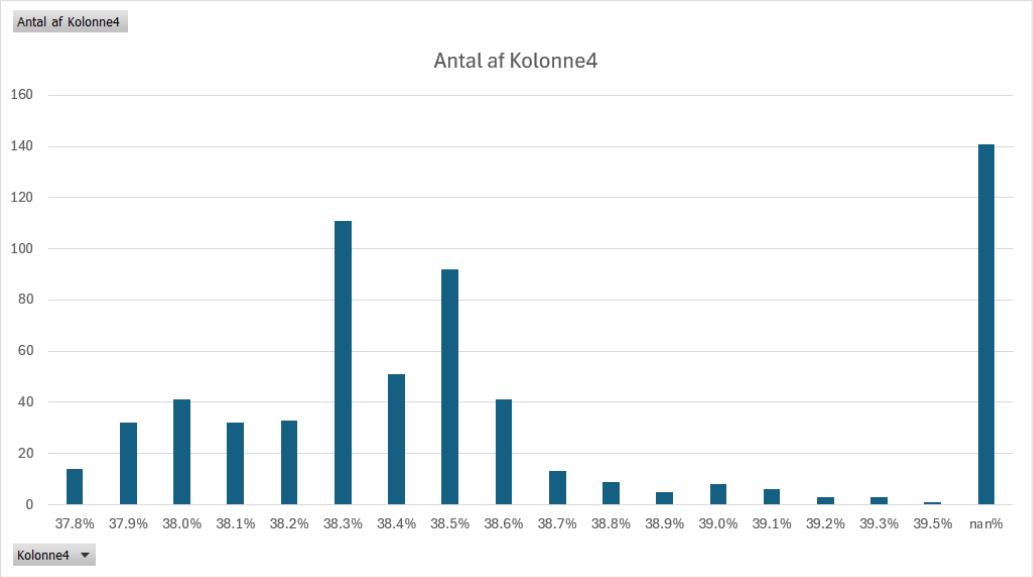
3. Deitel, P., & Deitel, H. (2023). *C how to program* (9th ed.). Pearson Education Limited.
4. Moodle.zealand.dk
5. debuggin kode!
6. https://www.mouser.com/datasheet/2/758/DHT11-Technical-Data-Sheet-Translated-Version-1143054.pdf
7. https://cdn.sparkfun.com/assets/f/7/d/9/c/DHT22.pdf
8. Jens-Erik Jensen (Biavler, Ringsted)
9. Ramsey, M.-T., Bencsik, M., Newton, M. I., Reyes, M., Pioz, M., Crauser, D., Simon Delso, N. & Le Conte, Y. (2020).
   "*The prediction of swarming in honeybee colonies using vibrational spectra*. Scientific Reports, 10:9798."
10. Ferrari, Silva, Guarino & Berckmans (2008)
    "*Monitoring of swarming sounds in bee hives for early detection of the swarming period"*
11. Draw.io
12. https://wokwi.com/projects/449786895943856129
13. https://token.com.tw/resistor/photo-cds.htm
14. https://components101.com/sites/default/files/component_datasheet/Micro-SD-Card-Module-Datasheet.pdf
15. https://www.analog.com/media/en/technical-documentation/data-sheets/ds3231.pdf
16. https://www.digikey.com/htmldatasheets/production/2047793/0/0/1/ssd1306.html
17. https://roboeq.ir/files/id/3882/name/KY-037.pdf/
18. https://github.com/Neas93/Datalogger
19. Vi havde 10 linjer med hvert sit kald og manuelt satte det sammen
    Prompt til chatgpt "Der må kunne forkortes til en mindre funktion"
20. Vi manglede en kodedel til at sammenligne værdierne
    Prompt til chatgpt " hvordan kan vi tage værdien fra KY-037 og få leveret en tekstværdi tilsvarende : lav, normal og høj ?"
21. Vi skulle bruge en måde at få tjekket hvorvidt den sidste SD-log er succesfuld
    Prompt til chatgpt " Hvordan kan jeg få vist på OLED hvorvidt SD log var ok?"

22. Vores DHT 22 sensor blev ved med at vise fejl : 999.99 måling. Herfter spugte vi chatten
    "Vi har denne sensor og har forsøgt at kigge på både software og hardware"
    Og den foreslog denne debugging kode

Bilag

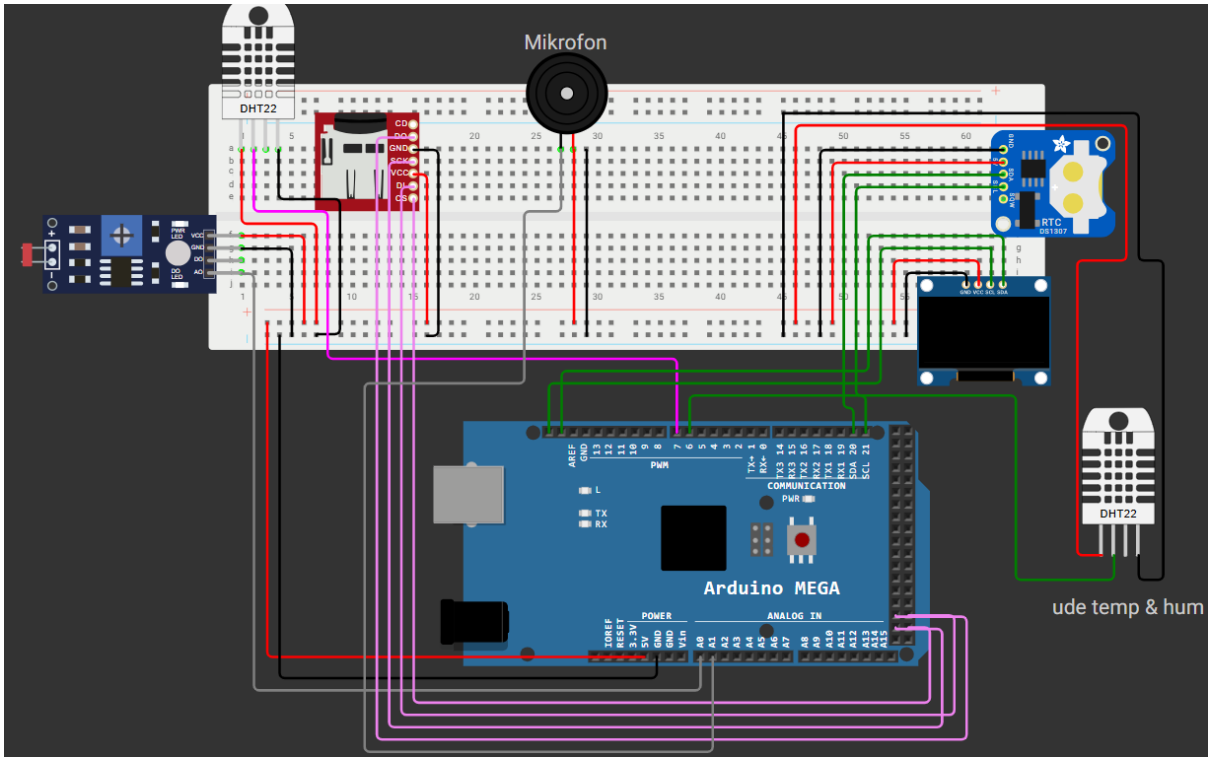
Bilag 1

Kolonne4	Antal af Kolonne4
37.8%	14
37.9%	32
38.0%	41
38.1%	32
38.2%	33
38.3%	111
38.4%	51
38.5%	92
38.6%	41
38.7%	13
38.8%	9
38.9%	5
39.0%	8
39.1%	6
39.2%	3
39.3%	3
39.5%	1
nan%	141

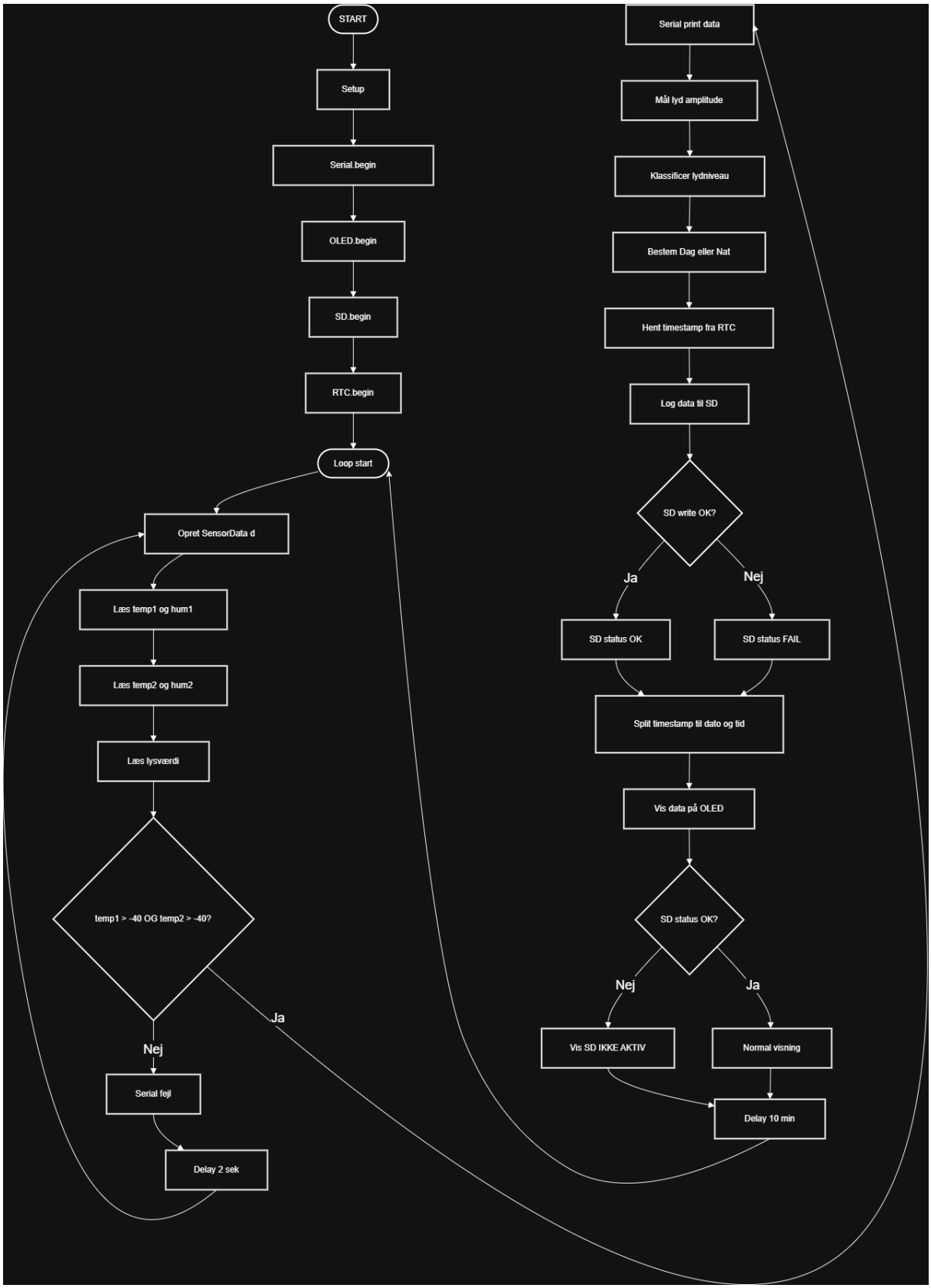


fejl %
22%

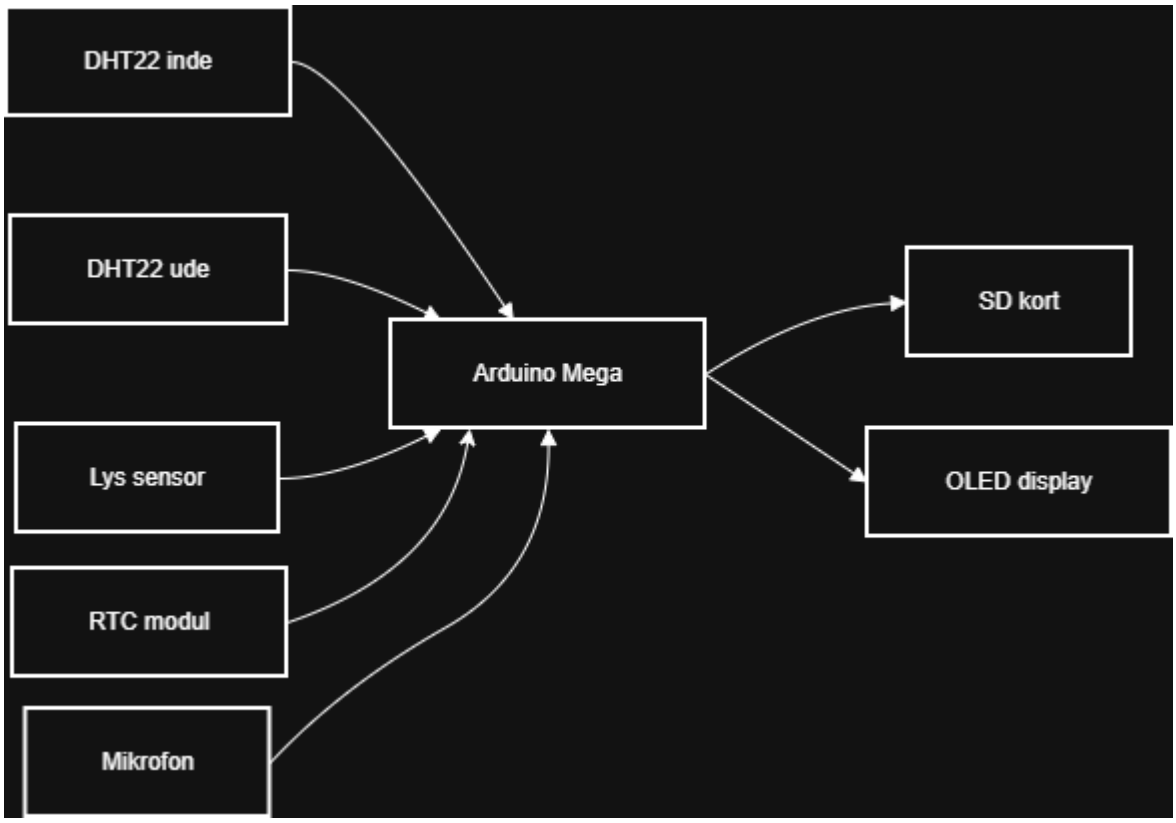
Bilag 2



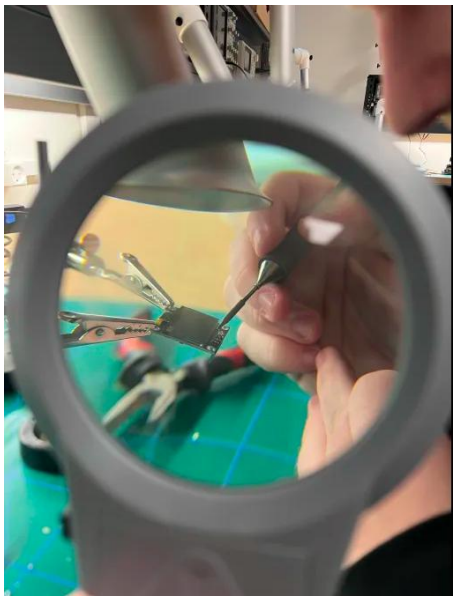
Bilag 3.


START
Setup
Serial.begin
OLED.begin
SD.begin
RTC.begin
Loop start
Opret SensorData d
Læs temp1 og hum1
Læs temp2 og hum2
Læs lysværdi
temp1 > -40 OG temp2 > -40?
Ja
Nej
Serial fejl
Delay 2 sek
Serial print data
Mål lyd amplitude
Klassificer lydniveau
Bestem Dag eller Nat
Hent timestamp fra RTC
Log data til SD
SD write OK?
Ja
Nej
SD status OK
SD status FAIL
Split timestamp til dato og tid
Vis data på OLED
SD status OK?
Nej
Ja
Vis SD IKKE AKTIV
Normal visning
Delay 10 min

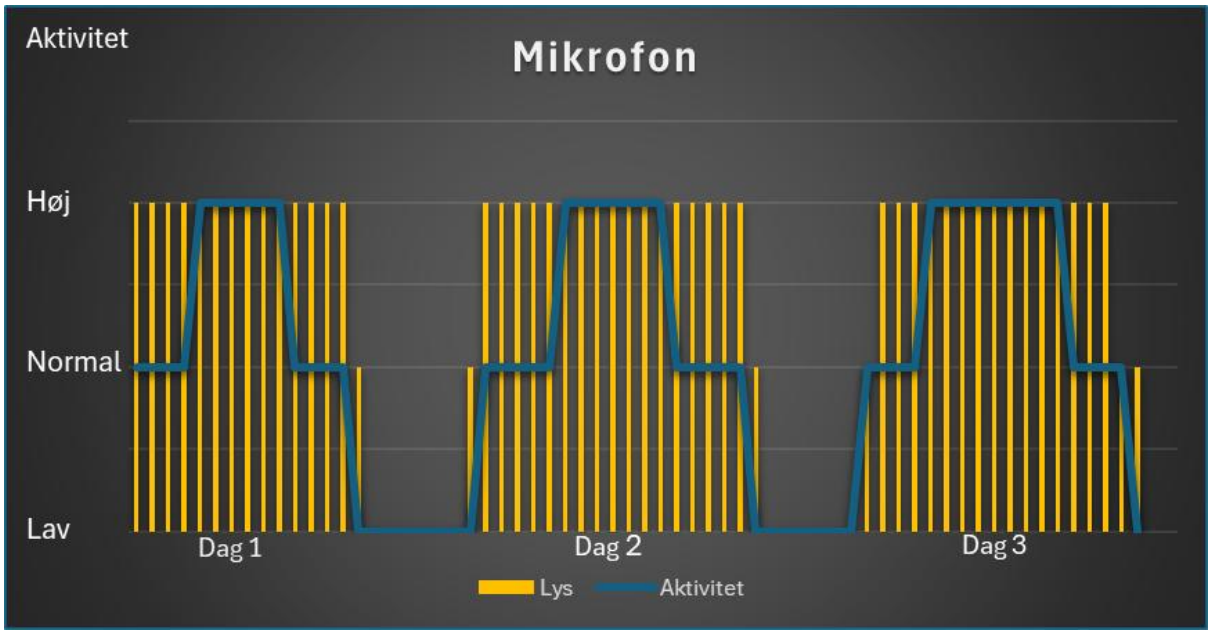
Bilag 4.



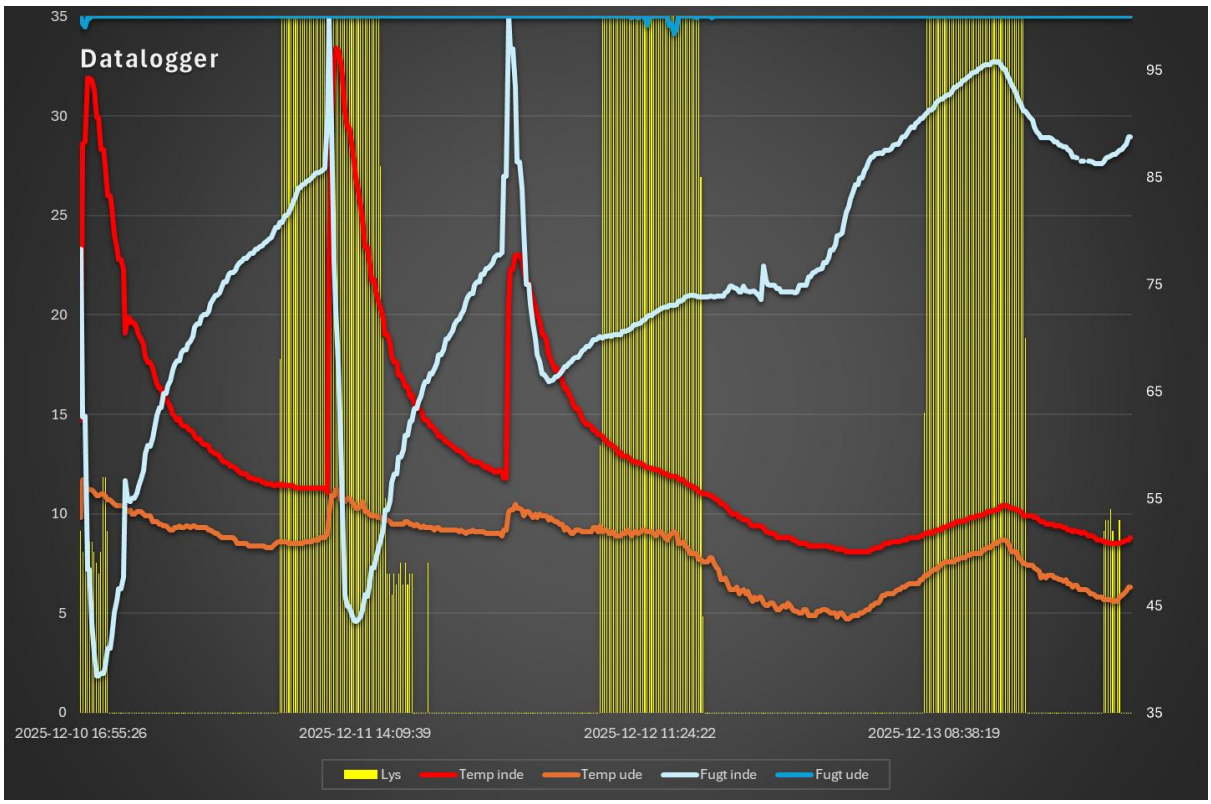
Bilag 5.



Bilag 6.



Bilag 7.



Bilag 8.

Samarbejdskontrakt

Datalogger gruppe Alpha

§1 Formål
Denne kontrakt har til formål at fastlægge rettigheder, pligter og forventninger for medlemmerne af projektgruppen for at sikre et professionelt og retfærdigt samarbejde.

§2 Gyldighed
Kontrakten er gældende fra underskriftsdato og indtil projektets afslutning, medmindre andet aftales skriftligt af alle medlemmer.

§3 Arbejdsindsats og engagement
Hvert medlem forpligter sig til aktiv deltagelse i projektets aktiviteter og møder. Opgaver, der er fordelt, skal udføres til tiden og i en kvalitet, der lever op til gruppens standard.
Manglende overholdelse uden gyldig grund betragtes som misligholdelse af kontrakten.

§4 Møder og kommunikation
Fremmøde til aftalte møder er obligatorisk. Gyldigt fravær (sygdom, force majeure) skal meldes inden mødet.
Meddelelser i gruppens discord chat skal besvares inden for 24 timer på hverdage.

§5 Roller og ansvar (Belbins rollemodel)

For at sikre et effektivt samarbejde er rollerne i projektgruppen fordelt efter Belbins rollemodel:

- **Lucas - Coordinator (CO)** – *Scrum master*
  Ansvarlig for planlægning, mødeledelse, opgavefordeling og overholdelse af deadlines.
- **Nicklas - Specialist (SP)** – *Teknisk ansvarlig*
  Ansvarlig for programmering, hardwareintegration og tekniske løsninger.
- **Muhammet - Monitor Evaluator (ME)** – *Analytisk ansvarlig*
  Ansvarlig for analyse, vurdering af løsninger, kvalitetssikring og faglig refleksion.

- **Oliver - Implementer (IMP)** – *Eksekverende ansvarlig*
  Ansvarlig for at samle delopgaver, sikre færdiggørelse og omsætte planer til konkrete resultater.

Rollerne er funktionelle og kan supplere hinanden efter behov, men det primære ansvar følger ovenstående fordeling.

§6 Deadlines
Interne deadlines fastsættes i fællesskab løbende og skal overholdes.

§7 Sanktioner ved misligholdelse
Første overtrædelse: Mundtlig påtale i gruppen.
Anden overtrædelse: Skriftlig advarsel underskrevet af alle gruppemedlemmer.
Gentagne eller alvorlige overtrædelser: Gruppen kan med fælles flertal indstille medlemmet til eksklusion fra samarbejdet. I sådanne tilfælde inddrages underviser, som træffer endelig afgørelse.

§8 Konflikthåndtering
Uenigheder søges løst internt gennem dialog og afstemning.
Hvis konflikten ikke kan løses, inddrages vejleder som mediator.

§9 Underskrifter
Ved at underskrive erklærer vi at vi har læst, forstået og accepteret ovenstående vilkår og sanktioner.

Dato
04/12 - 2025

Navn

Oliver Nicolai Bengtsson Muhammet Kart Nicklas Jensen Lucas Patrick Olsen

Bilag 9

Bilag 10